

播放感知的级联式流式语音对话上下文管理

第一作者
单位院系
单位名称
城市, 国家
邮箱

第二作者
单位院系
单位名称
城市, 国家
邮箱

Abstract—级联式架构（流式语音识别 → 大语言模型 → 流式语音合成）是当前语音对话系统的主流方案，但在用户打断（**barge-in**）场景下存在一个结构性的一致性缺陷：大语言模型的生成进度、语音合成进度与音频实际播放进度三者不同步，若系统将“已生成的全部内容”记入对话历史，模型便会在后续轮次引用用户从未听到的信息，造成对话错乱。本文实验表明，在朴素的生成位置截断策略下，此类未听内容引用在表面重叠口径下可达被打断轮次的 51%，在最保守的“特定信息引用”裁判口径下仍有 2.7%。“对话历史应等于用户实际听到的内容”这一原则已见于闭源商用系统，但学术界与开源社区尚无在大语言模型推理内部以显式 **KV** 缓存操作实现播放感知上下文管理的工作。针对上述问题，本文在形式化“生成—合成—播放”三进度指针与听到边界语义的基础上，设计并实现了三项机制：(1) 推测生成调度：以现成话轮检测模型输出的连续置信度配双阈值，将端点判断软化可为作废、可回滚的推测生成过程，以可控的冗余计算换取响应速度；(2) 播放感知的 **KV** 缓存管理：建立“播放采样—音频块—文本片段—token 区间”的反向映射时间轴，打断时经显式 **KV** 截断（注意力掩码与位置编码同步修正）与对话模板角色边界重建，使对话历史与用户所听内容严格一致；(3) 被打断历史的处理策略：朴素截断、打断标记与轻量模型重写三种策略的实现与消融。上述机制整合为首个开源、可复现的级联式参考实现。在 7B 主模型与 MultiWOZ 派生数据上的实验表明：播放感知截断使未听内容引用率在片段口径下未产生任何可感知的特定引用（0/24）；打断响应关键路径（反查 + 截断）恒为亚毫秒级且与上下文长度无关，**KV** 复用相对重新预填充在 8k 上下文处加速 39.7 倍；推测调度给出浪费率一延迟的连续可调权衡前沿，拐点处以约 4.5% 的 token 浪费将说完后首 token 延迟从 48.5 ms 降至 12.1 ms。本文同时提出“上界检测器—下界裁判—人工仲裁”三层一致性评测协议（人机一致性 $\kappa = 0.649$ ），对同类语音对话系统评测具有独立参考价值。

Index Terms—语音对话系统，打断，**KV** 缓存，推测生成，上下文一致性，低延迟

I. 绪论

A. 研究背景与动机

大语言模型（LLM）赋予语音助手前所未有的语义理解与生成能力，语音交互正从“指令式”走向“自然对话式”。在工程实践中，级联式架构（流式 ASR → LLM → 流式 TTS）因其模块可控、推理能力强，至今仍是语音对话系统的主流方案。

本文作者的前期工作（一期）针对级联架构的首字延迟问题，提出流式 ASR 与 LLM KV 缓存增量预填充的联合优化，使 TTFT 摆脱了随语音长度线性增长的约束。然而，延迟只是自然对话体验的一半；另一半是打断（**barge-in**）：人类对话中插话随时发生，系统被打断后能否“若无其事地”继续对话，取决于一个更隐蔽的问题——系统认为自己说了什么，与用户实际听到了什么，是否一致。

级联架构中这一问题具有结构性：LLM 的生成进度（token 级）快于 TTS 的合成进度（句子级），后者又快于音频的实际播放进度（受实时率约束）。当用户在播放中途打断时，若系统将“已生成的全部内容”记入对话历史，LLM 便会在后续轮次中引用用户从未听到的信息（“正如我刚才提到的……”），造成对话错乱。本文实验显示，在朴素截断策略下，此类“未听内容引用”在表面重叠口径下可达被打断轮次的 51%（第六章 E3）。

“对话历史应等于用户实际听到的内容”这一原则已见于 OpenAI Realtime API [1]、Azure Voice Live [2]、LiveKit Agents [3] 等商用系统（第二章详述）——本文不主张该原则的原创性。但这些实现均为闭源黑盒，且截断停留在受管理的转写文本层面；学术界与开源社区中，尚无工作在 LLM 推理内部以显式的 KV 缓存操作实现播放感知的上下文管理。本文旨在填补这一空白。

B. 问题与挑战

实现播放感知的上下文一致性，需在级联流水线内解决四个相互耦合的问题（第三章形式化）：

- 1) 跨模块反向映射：打断时系统只知播放到第几毫秒，而 KV 截断需要 token 级位置——需建立“播放采样 ↔ 音频块 ↔ 文本片段 ↔ token 区间”的四向映射并支持低延迟反查；
- 2) **KV** 截断合法性：截断后注意力掩码、位置编码须严格一致，且需重建对话模板的角色边界，多轮结构才合法；
- 3) 响应时机的软化：传统硬端点检测与“提前生成”矛盾——需以连续置信度的软触发驱动可作废的推测生成，以冗余计算换取响应速度，并管理作废时的 KV 回滚；
- 4) 被截断历史的语义完整性：截断落在句中时，历史里留下半句话，可能影响后续生成的连贯性。

C. 本文贡献

- **C1**（辅助）推测生成调度机制：以现成 turn-detection 模型输出的连续置信度配双阈值（推测/提交），将端点判断软化可为回滚的推测过程；在真实数据上给出推测浪费率与有效 TTFT 的九点权衡前沿，拐点处以约 4.5% 的 token 浪费将说完后延迟从 48.5 ms 压至 12.1 ms（第四章 §IV-B，第六章 E2/A3）。
- **C2**（核心）播放感知的 **KV** 缓存管理：反向映射时间轴 + 基于 **DynamicCache.crop** 的显式截断（掩码/位置编码同步）+ ChatML 角色边界重建 + 推测作废回滚的完整机制。打断响应关键路径（反查 + 截断）

实测恒为亚毫秒级且与上下文长度无关；相对重新预填充，KV 复用在 8k 上下文处加速 39.7 倍（第四章 §IV-C，第六章 E3/A1）。

- **C3**（扩展）被打断历史的处理策略：朴素截断/打断标记/轻量模型重写三种策略的实现与消融（第四章 §IV-D，第六章 A2）。
- 贯穿性贡献：上述机制的首个开源、可复现的级联式参考实现，以及一套含双口径一致性判定与人工仲裁协议的系统评测。

需要明确的边界：贡献 2 的高层理念（历史 = 听到内容）为商用系统先行实践，本文的创新在于开源可复现的机制级实现与可量化对比；软触发与重写模块均使用现成模型，不构成模型层贡献。

D. 论文组织结构

第二章梳理商用实践、学术研究与 KV 缓存技术三条脉络并定位本文；第三章形式化三进度指针、听到边界、截断合法性与全部评测指标；第四章给出三项机制的设计；第五章描述开源实现与部署；第六章报告延迟、权衡、一致性与消融实验；第七章讨论与商用系统的关系、局限与可推广性；第八章总结全文。

II. 相关工作

近年来，随着大语言模型赋予语音对话系统强大的语义理解与生成能力，级联式架构在工业界与学术界仍是主流方案。围绕“低延迟”与“自然打断”这两个核心诉求，相关研究可归纳为三条脉络：商用与工程系统中的打断-上下文管理实践、学术界的流式语音对话与打断处理方法、以及 LLM 推理层的 KV 缓存操作技术。本章依次梳理这三条脉络，并在此基础上界定本文工作的确切位置。

A. 商用与工程系统中的打断-上下文管理

在语音对话系统被用户打断时，一个根本性的问题是：已经生成、但用户并未完整听到的助手回复，应当以何种形式进入对话历史？若系统将“自己生成的全部内容”当作已说出的话记入上下文，则在后续轮次中 LLM 会误以为它表达了实际上并未传达给用户的信息，从而破坏多轮对话的一致性。

值得强调的是，“对话历史应当等于用户实际听到的内容”这一原则并非本文首次提出，已在多个商用系统中得到实现。OpenAI 的 Realtime API 提供 `conversation.item.truncate` 事件，客户端在检测到打断时传入 `audio_end_ms`（即音频实际播放到的位置），服务端据此删除未播放的音频及其对应的文本转写，其文档明确指出此举是“为确保上下文中不包含用户尚未听到的文本” [1]。微软 Azure 的 Voice Live 服务提供 `auto_truncate` 能力，在用户于播放期间开始说话时截断上一轮回复，并将会话上下文更新为已播放的部分；其官方文档几乎逐字表述了与上述相同的原则——“会话上下文应当被更新为反映用户实际听到的内容，否则 LLM 会假设它说了从未真正传达给用户的话” [2]。在开源工程框架层面，LiveKit Agents 在打断发生时仅将实际播放的转写文本提交进对话历史，并以 `interrupted=True` 标记该条被打断的消息，其文档说明转写会“被截断以匹配实际说出的输出” [3]。

因此，从“概念”层面看，播放感知的上下文截断已是成熟工程实践。然而，这些系统存在若干共同局限，恰为学术研究留下空间：其一，它们均为闭源黑盒（Realtime API、Voice Live）或仅在框架转发层做截断（LiveKit），未公开可复现的、面向 LLM 推理内部状态的实现；其二，截断的粒度停留在受管理的转写文本层面（删除 transcript 条目），而非对 LLM 的 KV 缓存做显式操作；其三，对“用户听到位置”的确定依赖近似估计——Azure 文档承认其“假设回复以实时速度播放”，OpenAI 则依赖客户端上报 `audio_end_ms`。需要审慎指出的是，这些差异是开源与闭源、显式 KV 操作与转写删除、测量与估计之别，而非“是否实现了该功能”之别：上述系统确实完成了播放感知的历史管理，本文不以“商用系统未做此事”作为立论依据。

B. 流式语音对话与打断处理的学术研究

学术界围绕级联式与端到端两类架构，从不同角度逼近低延迟自然交互，但均未涉及基于实际播放位置的 KV 缓存截断。

在级联式流式延迟优化方向，LTS-VoiceAgent [4] 提出 Listen-Think-Speak 框架，通过动态语义触发与增量推理降低“边想边说”的延迟，是与本文最接近的级联流水线工作；然而其全文未涉及打断、播放位置、KV 截断或对话角色边界重建，其“Pause-and-Repair”评测针对的是用户输入端的自我修正，而非 TTS 播放被打断的场景。RelayS2S [5] 采用双路架构，由快速的全双工语音到语音模型推测性地起草回复前缀并立即送入 TTS 以降低音频起始延迟，同时由慢速级联路径续写完整回复，并以一个轻量学习型验证器决定何时提交推测前缀；其“推测换延迟”的框架与本文的推测生成调度部分有重叠，但其目标是延迟起始而非打断作废，不处理 KV 缓存管理。此外，Amazon 的 Predictive ASR 等工作利用部分语音预测完整句子以提前触发 LLM 调用，同样聚焦触发时机而非上下文截断。

在端到端全双工方向，Moshi [6] 将用户与模型语音建模为并行的多流结构，取消显式的说话人轮次，使打断与重叠在架构层面自然涌现。由于其生成与真实时间帧同步、且不存在独立的 TTS 阶段，“生成即播放”隐式成立，因而从架构上回避了本文所针对的“生成进度、合成进度、播放进度三者不一致”问题——这从反面印证了级联式场景具有端到端模型所不具备的独特挑战。

在打断检测方向，FireRedChat [7] 通过流式个性化 VAD 抑制误触发的打断，对主说话人语音段打时间戳并在确认打断后立即暂停 TTS 播放；但其贡献停留在检测与时序控制层，并未处理已说出一半的助手回复如何进入历史，也不涉及 LLM 的 KV 状态。

C. LLM 推理与 KV 缓存操作

KV 缓存通过缓存历史 token 的键值状态、避免自回归生成中的重复计算，是 LLM 低延迟推理的基础技术。HuggingFace transformers 库提供 `DynamicCache` 等缓存抽象，其中 `crop` 方法支持将缓存截断到指定长度 [8]；vLLM、SGLang 等推理框架则围绕前缀缓存（prefix caching）与缓存复用做了系统级优化。这些机制主要面向上下文窗口溢出、滑动窗口限制与吞吐优化，并未面向”

由外部信号（如音频播放进度）驱动的动态截断与对话角色边界重建”这一场景。

与本文相关但正交的是若干文本 Agent 场景下的 KV 操作工作：IntentKV [9] 面向文本 LLM Agent 做跨轮意图感知的 KV 剪枝，Speculative Interaction Agents [10] 在工具调用被判定失效时丢弃推测性的工具观测结果。两者的截断均由文本/工具逻辑触发，与“用户在音频播放期间听到了多少”无关，也不涉及语音级联流水线。综上，在 transformers/vLLM 生态中，将 KV 缓存按播放进度显式 crop 并重建 ChatML 角色边界，作为一项系统/工程贡献，尚无已公开记录的开源先例。

D. 差异对比与本文定位

综合上述三条脉络，本文与最接近工作的差异如表 I 所示。

由表可见，播放感知的上下文截断在商用系统中已有实现，但均为闭源且停留在转写文本层面；而学术界与开源级联实现中，尚无工作在 LLM 推理内部以显式的 KV 缓存操作实现这一机制。本文的定位由此明确：面向开源、可复现的级联式 ASR→LLM→TTS 流水线，提供播放感知上下文一致性管理的完整实现，其技术核心是基于实际播放位置反向映射的显式 KV 缓存截断（DynamicCache.crop 配合 attention mask 与 position id 的同步重算）与 ChatML 角色边界重建。这一定位填补了表 I 中“开源、显式 KV 操作、级联”这一空白格，与商用系统形成“可复现的学术参考实现”与“闭源产品”的互补关系。

III. 问题形式化

本章将级联式流式语音对话中“生成—合成—播放”三进度不一致问题及全部评测指标形式化，为第四章方法与第六章实验提供统一符号体系。

A. 系统模型与符号定义

1) 级联流水线：考虑级联式流式语音对话系统 $\mathcal{S} = \langle \text{ASR}, \text{LLM}, \text{CHK}, \text{TTS}, \text{PLY} \rangle$ ，各阶段以流水线方式衔接：

- 流式 **ASR**：将用户语音流转写为确定性文本片段（final segment）序列 $U = \langle u_1, u_2, \dots \rangle$ 。片段一经产出即不再修正（区别于可撤销的 partial transcript），这是下游增量预填充可行的前提。
- **LLM**：维护对话上下文的 KV 缓存 $\mathcal{K}$ 。用户片段 u_i 到达即增量预填充（prefill）；回复以自回归方式逐 token 生成，产出 assistant token 序列 $Y = \langle y_1, \dots, y_G \rangle$ ，其中 G 为本轮生成总数。
- 断句器 **CHK**：将 token 流切分为句子片段（fragment）序列 $F = \langle f_1, \dots, f_m \rangle$ 。每个片段对应一个 token 区间

$$f_j \mapsto [\text{ts}(f_j), \text{te}(f_j)) \subseteq [0, G),$$

且 $\text{ts}(f_1) = 0$ 、 $\text{ts}(f_{j+1}) = \text{te}(f_j)$ ，即片段区间无缝衔接、共同覆盖 $[0, \text{te}(f_m))$ 。片段是本文截断操作的原子单位。

- 流式 **TTS**：以完整句子片段为输入、流式输出音频块（chunk）。片段 f_j 的音频占据累积播放时间轴上的采样区间 $[\text{ss}(f_j), \text{se}(f_j))$ （单位：采样点，采样率 r ）。
- 播放器 **PLY**：顺序播放音频块，并持续回报已播放采样数 $p \in \mathbb{N}$ （计数语义： p 表示采样区间 $[0, p)$ 已被用户听到）。

实现注记：ASR final 片段机制沿用一期工作（前缀/后缀滑窗）；片段-token 区间由非空白字符计数对齐算法维护，切分经验证严格无损。

2) 三个进度指针及其不一致性：在任一时刻 t ，系统中并存三个对同一回复的进度度量：生成指针 $g(t)$ （LLM 已生成的 token 数，token 级）、合成指针 $s(t)$ （TTS 已完成合成的最后片段的区间端点 $\text{te}(\cdot)$ ，片段级）、播放指针 $p(t)$ （已播放采样数，经反向映射可解析为 token 位置，采样级）。由于生成快于合成、合成快于实时播放，且播放器存在缓冲，三者满足单向滞后不变式：

$$p(t) \leq s(t) \leq g(t) \quad \forall t. \quad (1)$$

问题根源：当用户在播放中途打断（bargе-in）时，若系统以 g （已生成）或 s （已合成）为界决定“哪些内容进入对话历史”，历史中将包含用户从未听到的内容——后续轮次中模型可能引用它们（“正如我刚才提到的……”），造成用户感知不一致。式 (1) 表明该差距结构性存在于级联架构：端到端帧同步模型（如 Moshi）因“生成即播放”使三指针合一而回避此问题，级联架构则必须显式管理。三指针与听到边界的关系如图 1 所示。

B. 用户感知一致性

定义 1（听到边界）。设打断发生时播放指针为 p ，且 p 落于片段 f_k 的采样区间内（ $\text{ss}(f_k) < p \leq \text{se}(f_k)$ ）。听到边界定义为

$$H(p) = \text{te}(f_k), \quad (2)$$

即被打断的片段整体计入“已听到”（片段级向上取整）。当 $p = \text{se}(f_k)$ 时称干净边界（clean boundary），否则称半截片段（partial, $p < \text{se}(f_k)$ ）。

定义 2（用户感知一致性）。对话历史（及其 KV 表示）满足用户感知一致性，当且仅当被打断轮的 assistant 内容在历史中的保留范围恰为 $[0, H(p))$ ：

$$\mathcal{H}_{\text{ours}} = \langle y_1, \dots, y_{H(p)} \rangle. \quad (3)$$

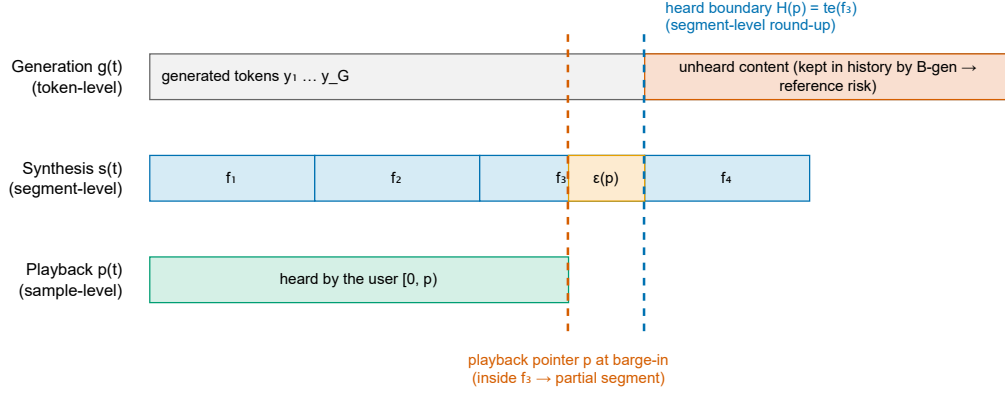
作为对照，朴素做法保留全部已生成内容 $\mathcal{H}_{\text{gen}} = \langle y_1, \dots, y_G \rangle$ ，其中 $y_{H(p)+1}, \dots, y_G$ 为用户未听到却进入历史的部分（第六章 E3 量化其危害）。

定义 3（片段粒度量化误差）。半截片段场景下，记 p^* 为 p 在片段 f_k 内按播放比例对应的 token 位置。选取片段级向上取整（定义 1）意味着未播出的片段尾部 $y_{p^*+1}, \dots, y_{H(p)}$ 计入历史但未被完整听到，称 $\varepsilon(p) = H(p) - p^*$ 为片段粒度量化误差，有界于单个片段长度： $\varepsilon(p) < \text{te}(f_k) - \text{ts}(f_k)$ 。它是“以片段为截断原子”这一工程取舍（换取截断实现的简洁与 TTS 韵律完整）的代价；本文不隐藏该误差，而在第六章以 strict 口径（定义 8）显式量化。干净边界处 $\varepsilon = 0$ ——实验中边界注入组 103/103 样本的 strict 残余全为零，验证该语义在真实数据上精确成立。

两类截断事件：(1) 播放期打断（bargе-in）： $p > 0$ ，按式 (3) 截断到 $H(p)$ ；(2) 推测作废（speculation invalidation）：推测生成尚未播出任何音频（ $p = 0$ ）时用户继续说话，整段推测输出作废，历史回滚到本轮 assistant 起点（等价于 $H = 0$ ）。

TABLE I
COMPARISON WITH THE CLOSEST RELATED WORK ON BARGE-IN CONTEXT MANAGEMENT

System / work	Truncation basis	Playback-aware	Context granularity	Cascaded	Open source
OpenAI Realtime API [1]	client-reported <code>audio_end_ms</code>	yes	managed transcript deletion	closed-box	no
Azure Voice Live [2]	real-time-rate assumption	yes (estimated)	conversation-context text	closed	no
LiveKit Agents [3]	actually-forwarded transcript	yes	word/sentence transcript, interrupted flag	framework	framework only (no KV)
RelayS2S [5]	no barge-in truncation	no	—	no (dual-path)	yes
LTS-VoiceAgent [4]	not covered	no	—	yes	—
FireRedChat [7]	detection-layer pause only	no	no LLM-context management	yes	partial
Moshi [6]	implicit by architecture	implicit	no separate TTS stage	no (end-to-end)	yes
Ours	measured playback $\rightarrow$ reverse map $\rightarrow$ token	yes	explicit KV crop + ChatML role rebuild, segment-level	yes	yes



Monotone-lag invariant: $p(t) \leq s(t) \leq g(t)$. B-ours keeps only $[0, H(p))$ in history; $\varepsilon(p)$ is the segment-granularity quantization error, bounded by one segment.

Fig. 1. Three progress pointers and the heard boundary. At barge-in the playback pointer p falls inside segment f_3 ; the heard boundary $H(p) = te(f_3)$ rounds up at segment granularity, and $\varepsilon(p)$ is the bounded quantization error. Content between p and g is unheard; keeping it in history (B-gen) creates reference risk.

C. 反向映射与截断合法性

定义 4 (反向映射). 给定播放采样数 p , 反向映射

$$\Phi: p \xrightarrow{\text{音频块}} f_k \xrightarrow{\text{片段}} [ts(f_k), te(f_k)] \xrightarrow{\text{token}} H(p)$$

将播放位置解析为 KV 截断点。Φ 须在打断到达时以近常数开销完成——它位于“打断即停”的关键路径上（第六章 A1 实测 Φ 与截断合计 0.31–0.34 ms, 与上下文长度无关）。支撑 Φ 的四向映射数据结构为第四章 §IV-C1 内容。

定义 5 (截断合法性). 设本轮 assistant 内容在 KV 中的起始位置为 a_0 , 截断至绝对位置 $N = a_0 + H(p)$. 截断操作合法, 当且仅当操作完成后: (1) 长度一致: KV 张量、注意力掩码与记录的序列长度三者相等 ($|K| = |\text{mask}| = N$); (2) 位置连续: 后续追加内容的位置编码自 N 起连续; (3) 角色合法: 在截断点重建对话角色边界 (注入 assistant 关闭、user 开启的模板标记), 使后续轮次的 prompt 结构与 chat template 一致。条件 (2)(3) 是工程实现的主要陷阱所在 (第四章 §IV-C2–§IV-C3 详述); 三项不变式的运行时检查贯穿本文全部实验。

D. 评测指标

以下指标均在确定性打断注入协议下测量: 打断以程序注入于受控播放比例 {25%, 50%, 75%} 及干净边界 (对照组), “用户听到的内容”以注入时刻的播放指针为真值, 实验完全可复现。

1) 延迟指标:

定义 6 (TTFT 双口径). $TTFT_{\text{text}}$: 软触发提交时刻 $\rightarrow$ LLM 首 token, 度量触发-生成链路本身; $TTFT_{\text{eff}}$: 用户

真实说完时刻 $\rightarrow$ 首个可用 token. 推测生成存活时已就绪 token 使 $TTFT_{\text{eff}} \approx 0$. 这是用户体感口径, 为本文主报告口径。

mouth-to-ear 延迟: 用户说完 $\rightarrow$ 听到首块音频, 建模为“首个句子片段就绪时刻 + TTS 首块合成延迟”, 其中 TTS 时序画像取实测值 (报告时注明建模属性与硬件条件)。

定义 7 (barge-in 响应延迟). 打断信号到达 $\rightarrow$ 播放停止且 KV 截断完成, 即 Φ 与截断的关键路径耗时。角色重建 (定义 5 条件 3) 不在关键路径上: 它可延迟至下一轮用户输入到达前执行, 不影响“打断即停”的体感。

2) 一致性指标:

定义 8 (未听内容引用率, loose/strict 双口径). 设打断轮“未听但入史”的文本为 W , 其后各轮回复为 R . 未听内容引用率为后续回复引用 W 中特有信息的样本比例。W 按两种口径: **loose** (片段级): W_{loose} 为完整未听片段中进入历史的部分。按定义 2, 本文方法下 $W_{\text{loose}} = \emptyset$, loose 引用率恒为零——这是机制的构造性保证, 而非实验发现; 该口径下实验量化的是对照方法的失败率。 **strict** (严格播放真值): W_{strict} 另含半截片段的未播尾部 (定义 3 的 ε 所对应文本), 量化片段粒度量化误差的实际后果。

引用判定采用双检测器并报告其一致性 (Cohen’s κ): 规则检测器基于词面线索匹配, 系统性偏高 (上界); LLM 裁判判定特有信息引用, 相对保守 (下界); 真值介于两者

TABLE II
METRIC-TO-EXPERIMENT MAPPING

Metric	Definition	Experiment
TTFT _{eff} , mouth-to-ear	Def. 6	E1
(ρ , TTFT _{eff}) frontier	Def. 9	E2/A3
Unheard-reference rate (loose/strict)	Def. 8	E3 (main)
Barge-in latency, KV-reuse gain	Def. 7	A1
History-policy coherence	Def. 8 (aux.)	A2

之间，另以人工小样本校验（第六章方法学讨论）。

3) 效率指标:

定义 9 (推测浪费率).

$$\rho = \frac{\sum \text{被作废的推测 token 数}}{\sum \text{被作废的推测 token 数} + \sum \text{最终生成 token 数}}. \quad (4)$$

KV 复用率: 截断事件中保留（复用）的 KV token 数占“复用 + 重算”之比；朴素截断与标记法下为 1，重写法下小于 1（被打断 assistant 段以重写文本重算）。

核心权衡曲线: 软触发推测阈值 θ 扫描下的 ($\rho(\theta)$, TTFT_{eff}(θ)) 前沿—— θ 越低（激进）浪费越高而延迟趋零，越高（保守）反之。该曲线刻画“以冗余计算换交互流畅度”的可调空间，是推测生成调度机制（贡献 1）的核心产出（第六章 E2）。

4) 指标—实验对应: 各指标与第六章实验的对应关系如表 II 所示。

IV. 方法设计

A. 总体设计

本文系统在一期“流式 ASR + LLM 增量 KV 预填充”流水线之上，新增输出侧三级流水（断句 → 流式 TTS → 播放）与两个控制机制，构成完整闭环，总体架构如图 2 所示。

设计遵循两条原则: (**P1**) 对话历史 = 用户实际听到的内容（该原则本身已见于商用系统，见第二章；本文给出其在开源级联栈上的显式 KV 机制实现）; (**P2**) 以冗余计算换交互流畅度，且冗余可测、可调 (§III 浪费率与权衡曲线)。

B. 推测生成调度（贡献 1）

1) 软触发的两阈值机制: 传统端点检测输出二值决策，误判即造成“抢答”或“迟钝”。本文以现成的 turn 检测模型（TEN Turn Detection [11]）输出连续置信度 $c_i = \text{conf}(u_1 \dots u_i) \in [0, 1]$ ，配两个阈值:

- 推测阈值 θ (激进): $c_i \geq \theta$ 即启动推测生成——注入 assistant 角色标记并预生成至多 B 个 token (B 为推测预算，限制单次作废成本；不用 κ 以免与 Cohen’s κ 混淆);
- 提交语义: 确定性协议下以用户真实说完为提交事件；实际部署中可另设保守阈值控制 TTS 播出时机（播出前的推测对用户不可见，作废零感知）。

软触发为文本侧模型，其推理与该片段的 KV prefill 并行执行（prefill 是既有开销），故触发判断不增加端到端延迟。

2) 推测-作废状态机: 对每个到达的用户片段 u_i : (1) 作废（若存在活跃推测）: 新片段到达意味着用户仍在说话、早前触发是误判 → 将 KV 截断回推测起点 b (user 内容末尾、assistant 标记之前)，推测产出计入浪费; (2) **prefill** u_i (user 角色保持打开，增量累积); (3) 评估 c_i ; $c_i \geq \theta$ → 记录推测起点 $b := |\mathcal{K}|$ ，注入 assistant 标记，预生成 $\leq B$ token 并缓存。

用户真实说完时: 若有存活推测，缓存 token 立即可用 (TTFT_{eff} ≈ 0)，继续生成余量；否则现场触发生成（支付全额 TTFT）。存活的推测正是“最后一个片段触发、其后再无语音”的那次——其计算被端点静默窗掩盖。作废的正确性依赖 §IV-C2 的合法截断: 推测回滚是“播放期打断”的退化情形 ($p = 0$ ，历史回滚到 $H = 0$)。

推测-作废状态机如图 3 所示。该机制把端点判断从硬决策转化为可调旋钮: θ 扫描给出 (ρ , TTFT_{eff}) 权衡前沿 (§VI-C)，系统按算力预算选择工作点。

C. 播放感知的 KV 缓存管理（贡献 2，核心）

1) 四向映射时间轴: 反向映射 Φ (定义 4) 由播放时间轴 (PlaybackTimeline) 支撑: 以片段记录为主轴的追加式结构，每条记录维护

(片段 f_j , token 区间[ts, te), 音频块列表, 采样区间[ss, se), 状态),

状态机为推测 → 合成中 → 已入队 → 播放中 → 已播完 / 已作废。三个生产者分别写入自己的映射维度（生成线程写 token ↔ 片段、TTS 线程写片段 ↔ 音频块并推进采样轴、播放线程原子更新已播采样数 p ），打断到达时反查按采样区间定位片段（计数语义 $ss < p \leq se$ ，边界处恰好判为“完整听完”）。片段数为轮级小量，全表单锁 + 播放游标无锁即满足亚毫秒反查 (§VI-E)。时间轴结构与反查过程如图 4 所示。

token 区间的来源: 断句器不感知 token。本文以非空白字符计数对齐建立句子片段与 token 区间的映射——向断句器供给 token 文本时累计非空白字符前缀和，片段产出时按其非空白字符数在前缀和上二分定位末 token。该映射对断句器的空白归一化免疫，且可验证无损（片段拼接与解码文本的非空白字符严格守恒）。

2) 合法的 KV 截断: 打断时刻， $\Phi(p)$ 给出听到边界 $H(p)$ ，KV 截断至绝对位置 $N = a_0 + H(p)$ 。满足定义 5 的三个条件需要同步维护三份状态: (1) KV 张量截断 (DynamicCache 的序列维切片 [8]); (2) 注意力掩码同步截短至 N ——掩码长度是后续 past length 的事实来源，仅截 KV 不截掩码会使位置编码整体错位（一期审查确认的首要陷阱）; (3) 序列长度与本轮 assistant token 账本同步更新，后续 prefill 的位置编码自 N 起显式重算。截断本身是张量切片（亚毫秒），构成“打断即停”的全部关键路径开销。

3) 角色边界重建: KV 是连续序列，角色信息仅存在于 chat template 的特殊标记中；截断落在 assistant 内容中间，直接续写会破坏 prompt 结构。本文从 tokenizer 的 chat template 推导出角色切换串（如 ChatML 的 `<|im_end|>\n<|im_start|>user\n`），截断后将其作为普通文本增量 prefill 进 KV——复用与流式 prefill 完全相同的显式掩码/位置编码机制，无需特殊算子。角色重建（一次 token 的小 prefill，数十毫秒）不在打断响应关键路径上: 打断只需“停播 + 截断”，重建可延迟到下一轮

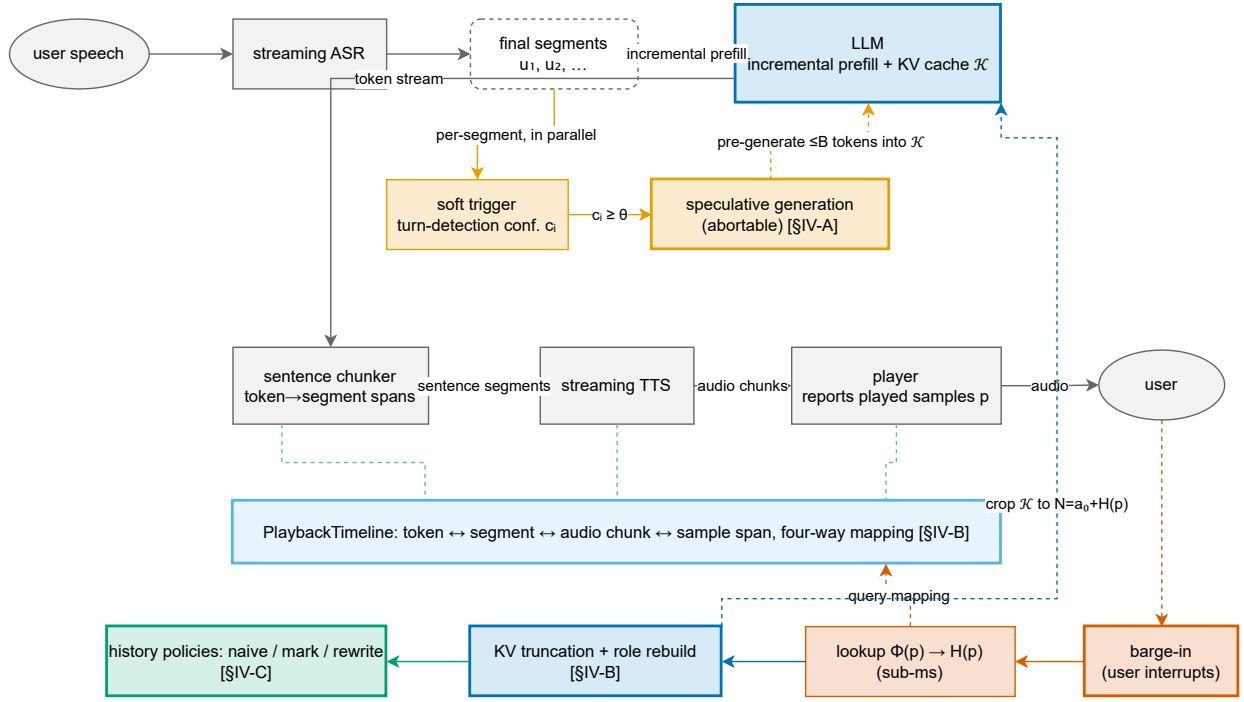
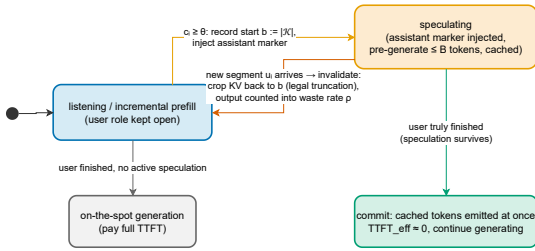


Fig. 2. System architecture. Top: input pipeline and soft-trigger-driven speculative generation (Sec. IV-B). Middle: output pipeline and the four-way mapping timeline (Sec. IV-C1). Bottom: barge-in chain—lookup $\Phi(p) \rightarrow H(p)$, KV truncation + role rebuild (Sec. IV-C), history policies (Sec. IV-D).



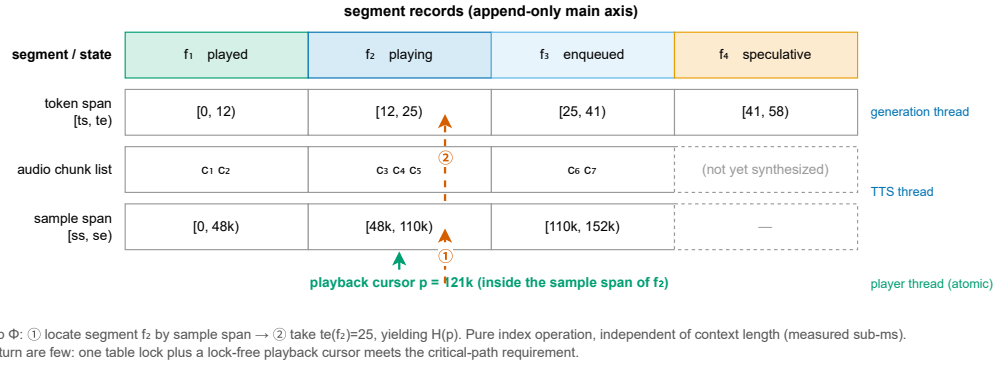


Fig. 4. The four-way mapping timeline and reverse lookup Φ . Segment records form the append-only main axis; three producer threads write the token-span, audio-chunk, and sample-span dimensions respectively. On barge-in, the segment is located by sample span and its $H(p)$ yields the heard boundary $H(p)$ —a pure index operation.

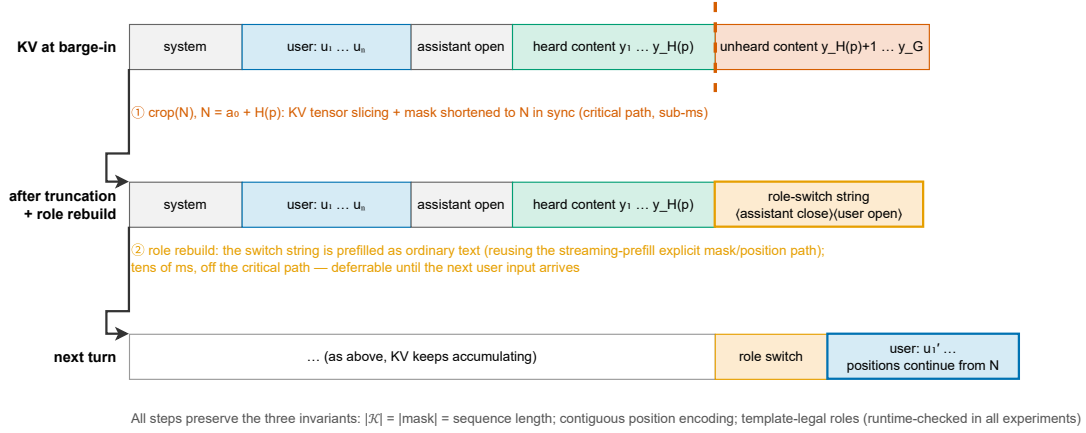


Fig. 5. KV truncation and role-boundary rebuild. ① At barge-in the cache is cropped along the heard boundary to $N = a_0 + H(p)$; unheard content leaves both KV and history—the critical path is this single sub-ms step. ② The role-switch string is prefill as ordinary text, off the critical path. ③ The next user turn continues with position encodings contiguous from N.

TABLE III
MODULE MAP OF THE OPEN-SOURCE IMPLEMENTATION

Module	Responsibility	Sec.
dialogue/timeline.py	four-way mapping timeline, reverse lookup Φ	§IV-C1
llm/stream_llm_inference.py (ext.)	truncatable KV (AccumKV-Cache), legal crop, role rebuild	§IV-C2–IV-C3
tts/	chunker + token-span alignment; StreamingTTS; CosyVoice2 adapter	§IV-C1
dialogue/trigger.py, rewriter.py	soft trigger; history rewriting	§IV-B, IV-D
dialogue/orchestrator.py	orchestration loop, barge-in chain, metric instrumentation	all
experiments/scripts/	6 experiment harnesses, calibration, LLM judge, TTS profiling	§VI

B. 关键实现点

可截断的 **assistant** 侧 **KV** 一期的生成循环不将新 token 的 KV 回写调用方，无法支持截断与多轮。本文扩展为：生成循环每步将 KV、注意力掩码、token id 同步写回统一容器（AccumKVCache，显式断言 DynamicCache

类型），并记录本轮 assistant 起点 a_0 与 token 账本——账本即时间轴 token 维的数据源。截断实现为 **DynamicCache.crop(N)** + 掩码切片 + 账本裁剪三者的原子组合；三份长度的一致性（定义 5 条件 1）以运行时不变式检查覆盖全部实验路径。

角色切换串的推导 不硬编码模板标记：初始化时以哑消息套用 chat template，差分提取”生成提示符”（user→assistant）并变换得到 assistant→user 切换串，兼容 ChatML 系模板；注入走与流式 prefill 相同的显式掩码/位置编码路径（§IV-C3）。

断句对齐的鲁棒性 非空白字符前缀和对齐（§IV-C1）之外的两个边界处理：纯空白句直接跳过（否则兜底推进会挪用下一片段首 token、使截断点偏移）；区间末端钳制到实际生成数（防越界截断）。半截片段的严格口径切分（strict）按播放采样比例切文本并吸附词边界，避免半词污染引用检测。

确定性实验协议 打断由 harness 程序注入（受控播放比例/片段边界），播放器为确定性时钟：TTS 时序画像（每字符采样数、首块延迟、RTF）取真机 benchmark 值驱动模拟——所有一致性/效率结论与真机时序等价且完全可复现；mouth-to-ear 为画像建模值并如实标注（§VI-D）。

TABLE IV
DEPLOYMENT CONFIGURATION

Role	Model	Size	Placement
Main LLM	Qwen2-7B-Instruct [14]	7B	GPU 0 (~14GB+KV)
Soft trigger	TEN Turn Detection [11]	7.6B	GPU 1 (~15GB)
History rewriter	Qwen3-0.6B [12]	0.6B	GPU 1
LLM judge (eval only)	Mistral-7B-v0.3 [15]	7B	GPU 1 (shared)
Streaming TTS	CosyVoice2 [16]	0.5B	GPU 1 (own env)

指标埋点 每轮落盘：8 个关键时间戳（说完/触发/首 token/首块/首播/打断注入/停播/截断完成）、双口径 TTFT、推测计数与浪费、KV 复用计数、时间轴快照（token $\leftrightarrow$ 片段 $\leftrightarrow$ 音频块 $\leftrightarrow$ 采样区间全表）——第三章每个指标都能从落盘数据复算。

C. 部署

部署配置如表 IV 所示。三个 LLM 实例独立部署、不复用权重，模拟真实多服务工程。全部模型配置经环境变量集中管理，验证环境（0.5B 开发替身）与实验环境切换零改码。

D. 验证方法学

代码可信度依三道防线建立：(i) 组件级 **smoke** 测试九套（时间轴边界语义、KV 三长度不变式、断句守恒、推测状态机、端到端打断链路等），全部随实验环境回归；(ii) 两轮对抗式代码审查——第一轮发现三个将污染实验数字的缺陷（E3 指标口径、E1 对照公平性、断句越界）并修复，第二轮验证修复并清理五个次要项，全程记录于决策日志；(iii) 数据级验证——干净边界注入组 103/103 样本 strict 残余为零 (§VI-B)，从实验数据侧印证截断语义实现正确。

VI. 实验与结果分析

A. 实验设置

硬件与部署 实验机为 2 $\times$ NVIDIA RTX 3090 (24 GB)：卡 0 部署主 LLM；卡 1 部署软触发、重写与裁判模型。全部延迟数字在该配置下测得。

模型 主 LLM 为 Qwen2-7B-Instruct [14]（与一期实验对齐）；软触发为 TEN Turn Detection [11]（7.6B，文本侧，输出 finished/unfinished/wait 类别词概率作为连续置信度，实测标定成对正确率 AUC= 1.00）；历史重写模型为 Qwen3-0.6B [12]；LLM 裁判为 Mistral-7B-Instruct-v0.3 [15]（与主 LLM 不同家族，避免同族偏袒）；流式 TTS 为 CosyVoice2-0.5B [16]。TTS 时序画像取真机实测：3175 采样/非空白字符（24 kHz，约 7.5 字符/秒语速）、首块合成延迟 2433.6 ms、合成实时率 RTF=0.513。需要说明，首块延迟为 3090 无 TensorRT 加速的实测值，官方报告 A100+TensorRT+fp16 下可至约 45 ms，§VI-D 据此做部署条件敏感性讨论。

数据 自 MultiWOZ 2.1 [17] 派生（固定随机种子）：103 条多轮对话（每条 ≥ 3 个 user 轮，用于 E3/A2）与 100 条子句切分话语（切分严格无损，段边界即停顿点，用于 E1/E2）。语种为英文。

打断注入协议 确定性程序注入 (§III)：对每条被打断轮，分别在播放比例 25%、50%、75%（多落于片段中间，构成半截片段）与片段干净边界（对照）注入打断，”用

TABLE V
UNHEARD-CONTENT REFERENCE RATE (E3 MAIN RESULT). B-OURS LOOSE 0% IS A CONSTRUCTIVE GUARANTEE; COHEN’S h : 1.59 = VERY LARGE, 0.47 = MEDIUM.

Criterion / detector	B-ours	B-gen	Fisher exact	h
loose / rule	0.0% (0/412)	51.0% (210/412)	1.8×10^{-79}	1.59
strict / rule	51.0% (210/412)	73.3% (302/412)	5.0×10^{-11}	0.47
loose / judge	0.0% (0/412)	2.7% (11/412)	9.1×10^{-4}	—
strict / judge	2.4% (10/412)	2.9% (12/412)	0.83 (n.s.)	—

户听到的内容”以注入时刻播放指针为真值，全部实验可复现。

被测条件 System A（非流式基线：说完 $\rightarrow$ 全量 prefill $\rightarrow$ 生成完 $\rightarrow$ 整段合成）；B-ours（本文方法：playback 截断）；B-gen（对照：generation 截断，历史保留全部已生成内容）。合成位置截断（B-syn）在同步合成模拟下与 B-gen 等价，本文不将其作为已验证条件。

B. E3: 多轮一致性（主结果）

每条件 $n=412$ （103 对话 $\times$ 4 注入位置）。表 V 给出未听内容引用率的双口径、双检测器结果。

主结论（写作红线：loose 口径下 B-ours 的 0% 是机制的构造性保证而非实验发现——历史中不存在未听片段，引用无从发生；实验量化的是对照方法的失败率）：

- 1) 朴素的 **generation** 截断在多数场景下引发未听内容复现：规则口径 51.0% 的后续回复复用了用户从未听到的词面内容；即使按最保守的裁判口径（仅特定信息引用），仍有 2.7% 的轮次让模型”引用了自己从未说出口的话”，两把尺子下差异均显著 ($p < 10^{-3}$)。
- 2) 打断越早，危害越大：B-gen 的 loose 引用率随注入位置单调递减——25% 处 85.4% (88/103)、50% 处 48.5%、75% 处 20.4%；干净边界处 49.5%。未听内容越多，被复现的概率越高，与直觉一致，如图 6 所示。
- 3) 片段粒度量化误差的后果可忽略：strict 口径下 B-ours 存在半截片段尾部残余（定义 3），规则检测器报 51.0%——但这是词面重叠上界；裁判口径下 B-ours strict 仅 2.4%，且与 B-gen (2.9%) 无显著差异 ($p = 0.83$)。即：以片段为截断原子的工程取舍，在”特定信息被错误引用”的意义上没有可测量的额外代价。
- 4) 截断语义精确成立：干净边界注入组 103/103 样本的 strict 残余为零，与定义 1 的边界语义一致，可视作为机制实现正确性的数据级验证。

检测器方法学 规则检测器与 LLM 裁判在真实数据上一致性很低 (Cohen’s $\kappa \approx 0.05$ ；开发用小样本上为 0.71)。原因是 MultiWOZ 任务域高频词 (hotel、book、train 等) 使词面线索在域内自然复现，规则检测器系统性过触发。因此本文将规则口径解释为表面重叠上界、裁判口径为特定引用下界，真值介于其间；关键在于两把尺子下 **B-ours** 均不劣于 **B-gen**，且 **loose** 口径双双为零，主结论对检测器选择鲁棒。

为仲裁两把尺子，按判定组合分层抽取 37 条样本进行人工标注（判定标准与裁判一致：仅特定事实/数字/措辞的引用计为”是”）。结果（表 VI）：人工与裁判在 loose 口

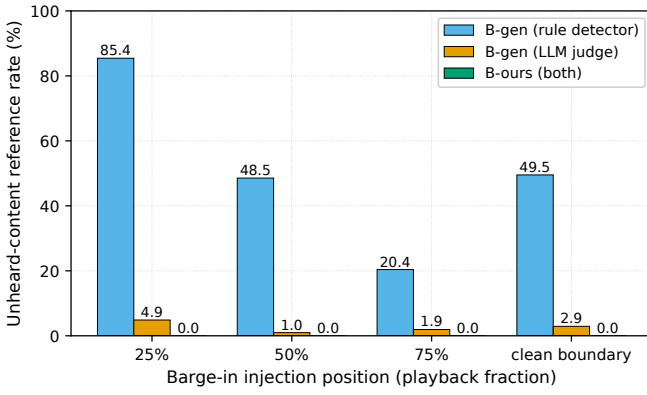


Fig. 6. Unheard-content reference rate by injection position. B-gen decreases monotonically with later interruption; B-ours is zero under both criteria—a constructive guarantee of the mechanism, not a statistical finding.

TABLE VI
HUMAN ARBITRATION ($n=37$, STRATIFIED OVER JUDGMENT COMBINATIONS)

Comparison	Criterion	Agreement	Cohen's
Human vs. LLM judge	loose ($n=13$)	84.6%	0.649
Human vs. rule detector	loose ($n=13$)	30.8%	-0.073
Human "specific reference"	strict ($n=24$)	—	0/24

径中等偏强一致（一致率 84.6%， $\kappa = 0.649$ ），而与规则检测器的一致性不优于随机（一致率 30.8%， $\kappa = -0.073$ ）——支持以裁判口径为正文主数字、规则口径仅作上界呈现。尤为重要的是，**strict** 口径的 **24** 条样本中人工判定无一构成特定引用（**0/24**）：片段级截断粒度引入的量化误差尾部（定义 3）在人类判定下不产生可感知的一致性损害，为第四章“以片段为截断单位”的设计取舍提供了实证辩护。

C. E2: 推测浪费率- $TTFT_{\text{eff}}$ 权衡曲线（核心图）

TEN 软触发阈值按实测置信度分布标定后扫描九点（含“永不推测”哨兵）。结果见表 VII 与图 7。

分析 曲线呈单调权衡且存在明显拐点区（ $\theta \in [0.85, 0.97]$ ）： $\theta = 0.92$ 时以 4.5% 的冗余计算换取 $TTFT_{\text{eff}}$ 从 48.5 ms 降至 12.1 ms（-75%）； $\theta = 0.969$ 时浪费率仅 0.8% 仍获 40% 的延迟收益。这验证了 §III 的设计命题——端点判断从硬决策变为可调的连续旋钮，系统可按算力预算选择工作点。激进端（ $\theta \leq 0.4$ ）浪费率 14-29% 而 $TTFT_{\text{eff}} \approx 0$ ：推测几乎总在用户说完前就绪。注意 7B 模型下“不推测”的 $TTFT$ 全额仅 48.5 ms（得益于一期流式 prefill 已消除上下文编码开销），故本实验的收益比例（-75% ~ -99%）比绝对毫秒数更具外推意义；对更大模型或更长回复前缀，绝对收益将同步放大。本表 records 级数据同时构成消融 A3（激进度扫描）。

D. E1: 端到端延迟

$n=100$, B-ours 取 $\theta = 0.391$ 工作点。 $TTFT$ 口径（真实测量）：System A 平均 27.4 ms，B-ours $TTFT_{\text{eff}}$ 平均 **0.6 ms**（改善 97.9%）——推测存活使 token 在用户说完时几乎总已就绪。

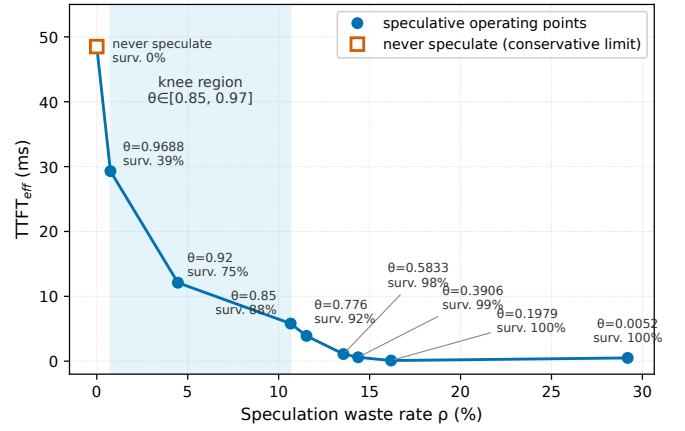


Fig. 7. Speculation waste rate vs. $TTFT_{\text{eff}}$: a continuously tunable frontier; the open square is the never-speculate conservative limit.

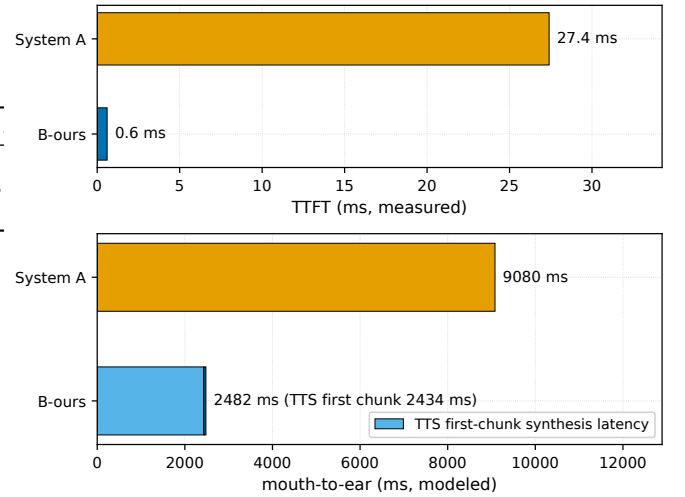


Fig. 8. End-to-end latency. Top: $TTFT$ (measured). Bottom: mouth-to-ear (modeled: first-segment ready + TTS first chunk, RTX 3090 profile); of B-ours' 2482 ms, 2434 ms is TTS first-chunk synthesis latency.

mouth-to-ear 口径（建模值：首片段就绪时刻 + TTS 首块延迟，画像为 3090 实测）：A 为 9080 ms（须等全部生成 + 整段合成），B-ours 为 2482 ms，其中 2434 ms 由 TTS 首块合成延迟主导。部署条件敏感性：该首块延迟强依赖推理加速条件——按官方 A100+TensorRT 报告值（ ≈ 45 ms）外推，B-ours 的 mouth-to-ear 可至约 100 ms 量级，而 System A 的“等全部生成 + 整段合成”是结构性成本、随回复长度线性增长，不随 TTS 加速消失。如图 8 所示，建模量与实测量分别标注。

E. A1: KV 复用收益与打断响应延迟（7B）

截断路径与重新 prefill 的对比如表 VIII 与图 9 所示。**barge-in** 响应延迟恒为亚毫秒级且与上下文长度无关——反查（定义 4 的 Φ ）为纯索引操作、截断为张量切片；角色重建可延迟到下一轮输入前执行，不在“打断即停”的关键路径上。作为对照，放弃 KV 复用而对听到边界前的完整上下文重新 prefill 的成本随长度线性增长，8k 上

TABLE VII
SOFT-TRIGGER THRESHOLD SWEEP ($n=100$ PER POINT)

Threshold θ	0.005	0.198	0.391	0.583	0.776	0.85	0.92	0.969	1.1 (never)
Waste rate ρ	29.2%	16.2%	14.4%	13.6%	11.5%	10.7%	4.5%	0.8%	0%
TTFT _{eff} (ms)	0.5	0.1	0.6	1.1	3.9	5.8	12.1	29.3	48.5
Survival rate	100%	100%	99%	98%	92%	88%	75%	39%	0%

TABLE VIII
TRUNCATION PATH VS. RE-PREFILL (MEDIAN, MS)

Context length (tokens)	275	1k	2k	4k	8k
Lookup + crop (critical path)	0.31–0.34 (near-constant)				
Role rebuild (off critical path)	25–47				
Re-prefill	72	235	459	891	1863
Speedup	2.8×	8.6×	15.3×	25.2×	39.7×

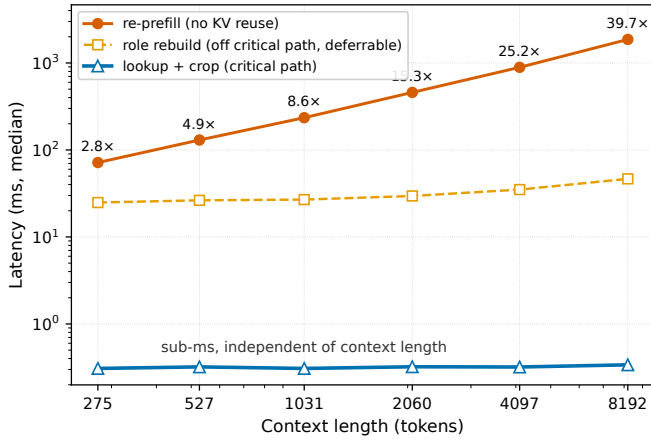


Fig. 9. Barge-in response paths vs. re-prefill across context lengths (log-log). Lookup+crop is sub-ms and length-independent; role rebuild is off the critical path; re-prefill grows nearly linearly (39.7× at 8k).

下文时达 1.86s——多轮语音对话的上下文天然持续增长，KV 复用收益随对话进行单调扩大。

F. A2: 被打断历史的三种处理策略

$n=100$ /策略，裁判连贯性评分 (1–5)：朴素截断 3.76、重写法 3.62、标记法 3.29；重写延迟 mean 639 ms / P90 937 ms / max 1165 ms。

诚实报告：重写法未优于朴素截断 (Wilcoxon 配对检验 $p = 0.32$ ，无显著差异)，标记法反而更低。可能原因：(i) 7B 主模型对半句历史已有较强鲁棒性，重写收益空间小；(ii) 0.6B 重写模型的改写质量有限；(iii) 单裁判单点评分的敏感度不足。工程侧结论仍成立——重写延迟 $P90 < 1s$ ，可隐藏于用户打断后的说话期内 (架构上并行执行)，且 KV 复用率按重算段比例可控。据此，本文将历史自然化重写定位为探索性结果：机制可行、延迟可隐藏，但在本实验设置下未观察到一致性收益，留待更强重写模型与更细评价方式 (如成对偏好) 检验。

G. 结果小结

- 一致性 (主张核心)：playback 截断使历史与用户所听严格一致 (loose 恒 0，构造性)，对照方法在 51% (词面) / 2.7% (特定引用) 的场景引入未听内容复现；片段粒度量化误差在特定引用意义上无可测量代价 ($p = 0.83$)。
- 延迟：推测生成把 TTFT_{eff} 压至亚毫秒-数毫秒并可按 $(\rho, TTFT)$ 曲线调节 (拐点 $\theta \approx 0.92$ ：4.5% 浪费换 75% 延迟削减)；打断响应亚毫秒且与上下文无关；KV 复用相对重 prefill 最高 39.7×
- 方法学：双检测器夹逼 + 干净边界 103/103 零残余的实现验证，构成结果可信度的两道防线；重写策略如实报告为无显著收益的探索性结果。

VII. 讨论

A. 与商用系统的关系

本文与 OpenAI Realtime、Azure Voice Live、LiveKit 的关系是开源参考实现与闭源产品的互补，而非能力有无之别 (第二章表 I)。差异集中在三处：其一，操作层次——商用系统删除受管理的转写条目，本文在 LLM 推理内部对 KV 缓存做显式截断，保留了跨轮 KV 复用带来的延迟收益 (A1: 8k 上下文处 39.7×)；其二，位置来源——Azure 以“实时速度假设”估算、OpenAI 依赖客户端上报，本文由播放器回报采样计数并经反向映射精确到片段边界；其三，可检视性——本文的映射表、时间戳与截断决策全部落盘，实验可复现。需要强调的是，这些差异不支持“商用系统未实现该功能”的论断——它们实现了，且其转写级方案在其产品约束下是合理的工程选择。

B. 局限与效度威胁

一致性度量的仪器差异。规则检测器在任务域 (MultiWOZ 高频词) 上系统性过触发，与人工仲裁一致性不优于随机 ($\kappa = -0.073$)；LLM 裁判获得中等偏强的人工一致 ($\kappa = 0.649$) 但样本量有限 (37 条)。本文以“上界—下界—仲裁”三层协议缓解，但更大规模的人工标注将进一步收紧区间。

重写策略未显收益。A2 中 0.6B 重写模型的裁判连贯性 (3.62) 未超过朴素截断 (3.76)。可能原因：7B 主模型对半句历史本就鲁棒；0.6B 重写引入的措辞变化反而稀释了上下文线索。这提示贡献 3 的价值依赖于主模型与重写模型的能力配比，作为扩展贡献如实报告。

时序建模的等价性假设。开发与部分实验以实测时长画像驱动的 Mock TTS 进行，其与真机的等价性仅限时序维度；mouth-to-ear 的正式数值来自真机画像 (3090 无 TRT 首块 2434 ms)，与官方 A100+TRT 指标 (45 ms) 相差两个数量级——硬件与推理栈对该指标影响极大，跨系统比较需谨慎。

数据与语种范围。实验限于英文任务型对话 (MultiWOZ 派生) 与程序注入的确定性打断; 开放域对话、真实用户打断行为与中文场景 (stanza 断句路径、CosyVoice2 中文合成) 留待扩展。打断时机的固定比例注入覆盖受控但非自然分布。

软触发的域适配。TEN 在本文句式上可分性极强 (标定 AUC=1.00), E2 的浪费主要来自派生数据中“句法近似完整”的假停顿; 口语更强的犹豫、修正现象下阈值前沿的形状可能不同。

C. 可推广性

机制层面的三个组件均不绑定具体模型栈: 反向映射表只要求下游产出“片段 → 音频块”的归属关系, 任何句子级输入的流式 TTS 均可接入 (StreamingTTS 接口); KV 截断与角色重建依赖 transformers 的 DynamicCache 与 ChatML 类模板, 适用于该生态的任意因果 LM; 软触发仅需一个输出连续置信度的文本判断器。三进度指针的不变式 (1) 与听到边界语义 (定义 1) 对任何“生成快于播放”的级联系统成立, 包括未来接入更低延迟 TTS 或更大主模型的配置。

VIII. 总结与展望

A. 全文总结

本文面向级联式流式语音对话系统的打断场景, 研究“系统所记的对话历史”与“用户实际听到内容”的一致性问题。在形式化“生成—合成—播放”三进度指针与听到边界语义的基础上, 本文实现并验证了三项机制: 以连续置信度双阈值驱动的推测生成调度 (C1)、基于反向映射与显式 KV 截断的播放感知上下文管理 (C2)、以及被打断历史的三种处理策略 (C3), 并将其整合为首个开源、可复现的级联式参考实现。

实验 (7B 主模型、MultiWOZ 派生数据、专用 turn-detection 模型) 表明: 打断响应关键路径恒为亚毫秒级且与上下文长度无关, KV 复用相对重新预填充在 8k 上下文处加速 39.7 倍 (A1); 推测调度以约 4.5% 的 token 浪费将说完后的首 token 延迟从 48.5 ms 降至 12.1 ms, 权衡前沿连续可调 (E2); 在一致性上, 播放感知截断将片段口径的未听内容引用率由构造保证为零, LLM 裁判口径下对照组仍有 2.7% 的特定引用与 51% 的表面重叠上界, 人工仲裁 ($\kappa = 0.649$) 支持裁判口径的可信度, 且片段级截断粒度的量化误差在人工判定下未产生任何可感知引用 (E3)。

方法学上, 本文提出的“上界检测器—下界裁判—人工仲裁”三层一致性判定协议, 以及对规则检测器在任务域过触发现象的定量揭示, 对同类语音对话评测具有独立参考价值。

B. 展望

- 更细截断粒度: 在片段级机制之上探索词级甚至 token 级听到边界 (需 TTS 提供词级时间戳), 并量化其相对片段粒度的边际收益——本文 strict 口径 0/24 的人工判定提示该收益可能有限;
- 位置估计方式对比: 量化“实测播放缓冲”与商用系统“实时速度假设”在拥塞、变速等非理想播放条件下的一致性差异;

- 真实音频闭环与中文场景: 接入真实用户语音与流式 ASR 的全链路、扩展 CrossWOZ/中文合成路径;
- 端到端与级联的融合: 在保留级联可控性的前提下, 借鉴帧同步思想进一步压缩三指针间隙。

一期工作打破了“延迟随语音长度线性增长”的约束, 本文进一步回答了“被打断之后如何保持对话一致”的问题。两者共同表明: 在可预见的将来, 级联架构仍可通过细粒度的流式状态管理, 不断逼近端到端系统的交互自然度, 同时保有其模块化与推理能力优势。

致谢

本文写作过程中使用大语言模型辅助文献检索核验、绘图脚本与语言润色; 全部技术内容、实验与结论由作者完成并核验。

REFERENCES

- [1] OpenAI, “Realtime API — conversations,” <https://developers.openai.com/api/docs/guides/realtime-conversations>, 2025, accessed: 2026-07-21.
- [2] Microsoft Azure, “Voice Live auto-truncation,” <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/how-to-voice-live-auto-truncation>, 2025, accessed: 2026-07-21.
- [3] LiveKit, “Agents — events and interruption handling,” <https://docs.livekit.io/agents/build/events/>, 2025, accessed: 2026-07-21.
- [4] W. Zou, Y. Miao, Z. Ma, J. Xu *et al.*, “LTS-VoiceAgent: A listen-think-speak framework for efficient streaming voice interaction via semantic triggering and incremental reasoning,” 2026, arXiv:2601.19952.
- [5] L. Mai, “RelayS2S: A dual-path speculative generation for real-time dialogue,” 2026, arXiv:2603.23346.
- [6] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour, “Moshi: A speech-text foundation model for real-time dialogue,” 2024, arXiv:2410.00037.
- [7] FireRed Team, “FireRedChat: Streaming personalized VAD for barge-in,” 2025, arXiv:2509.06502.
- [8] Hugging Face, “Transformers documentation: KV cache strategies,” https://huggingface.co/docs/transformers/en/kv_cache, 2025, accessed: 2026-07-21.
- [9] J. Li, J. Lou, and J. Li, “IntentKV: Cross-turn intent-aware KV cache pruning for agent inference,” 2026, arXiv:2606.09916.
- [10] C. Hooper, M. Kang, S. Moon *et al.*, “Speculative interaction agents: Building real-time agents with asynchronous I/O and speculative tool calling,” 2026, arXiv:2605.13360.
- [11] TEN Framework, “TEN turn detection: Turn detection for full-duplex dialogue communication,” <https://github.com/TEN-framework/ten-turn-detection>, 2025, accessed: 2026-07-21.
- [12] A. Yang *et al.*, “Qwen3 technical report,” 2025, arXiv:2505.09388.
- [13] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proc. 40th Int. Conf. Machine Learning (ICML)*, 2023, pp. 28 492–28 518.
- [14] A. Yang *et al.*, “Qwen2 technical report,” 2024, arXiv:2407.10671.
- [15] A. Q. Jiang, A. Sablayrolles, A. Mensch *et al.*, “Mistral 7B,” 2023, arXiv:2310.06825.
- [16] Z. Du *et al.*, “CosyVoice 2: Scalable streaming speech synthesis with large language models,” 2024, arXiv:2412.10117.
- [17] M. Eric, R. Goel, S. Paul, A. Sethi, S. Agarwal, S. Gao, A. Kumar, A. Goyal, P. Ku, and D. Hakkani-Tür, “MultiWOZ 2.1: A consolidated multi-domain dialogue dataset with state corrections and state tracking baselines,” in *Proc. 12th Language Resources and Evaluation Conf. (LREC)*, 2020, pp. 422–428.